a bit clearer

C PROGRAMMING FROM SCRATCH · EPISODE 15

Arithmetic Operators

Doing math in C, from 9 / 2 to ++

KEY POINTS

- When both operands of / are integers, C does integer division and drops the remainder. Make one operand floating-point to get a decimal.
- The % operator gives the remainder of two integers. Its sign follows the first operand, and % 10 gives the last digit.
- scanf reads input into variables through their addresses (&num1). In Visual Studio, put #define _CRT_SECURE_NO_WARNINGS 1 on the very first line.
- ++ and -- add or subtract one. In front (++n) they change first; behind (n++) they use first.

01 Arithmetic operators

▶ VIDEO 00:51

`+ - * / %` are **binary** operators: each takes a left and a right operand. Multiplication is `*`, division is `/`. With `a = 10` and `b = 22`: `a + b` is `32`, `a - b` is `-12`, `a * b` is `220` (`add-sub-mul.c`).

02 Integer division

▶ VIDEO 02:16

division.c — C SOURCE

```c
float z = 9 / 2;      /* 4.000000 */
z = 9.0f / 2;         /* 4.500000 */
```

KEY IDEA

When both sides of `/` are integers, the result is an integer: `9 / 2` is `4` before it ever reaches `z`. Make at least one operand floating-point (`9.0f / 2`) to get `4.5`.

WATCH OUT

With `score = 5`, `score = score / 20 * 100;` gives `0`: `5 / 20` is already `0`. Use `20.0` and a `double`: `25.000000`.

03 The remainder operator %

▶ VIDEO 04:22

EXPRESSION	RESULT	
`9 % 2`	`1`	what `9 / 2` drops
`11 % -5` · `-11 % -5` · `-11 % 5`	`1` · `-1` · `-1`	the sign follows the first operand
`123 % 10` · `123 % 100`	`3` · `23`	`% 10` gives the last digit

`%` only works with integers: on a `float` it fails with `error C2296`.

04 Reading input with scanf

▶ VIDEO 05:37

average.c — C SOURCE

```c
#define _CRT_SECURE_NO_WARNINGS 1
#include <stdio.h>

int main()
{
    int num1 = 0;
    int num2 = 0;
    scanf("%d %d", &num1, &num2);

    double avg = (num1 + num2) / 2.0;
    printf("%lf\n", avg);
    printf("%.1lf\n", avg);
    return 0;
}
```

- `&num1` is the **address** of `num1`: where scanf puts the value. Placeholders match the type: `%d` int, `%f` float, `%c` char, `%lf` **double**.
- Type a space between the numbers (`10 5`); `105` is read as one number. `%.1lf` prints `7.5`.
- Without line 1, Visual Studio stops with `error C4996: 'scanf': This function or variable may be unsafe.` The define must be the **very first line**, above every `#include`. `scanf_s` is mainly Microsoft's version; GCC may not support it. To start every new file with the line, add it to Visual Studio's template `Common7\IDE\VC\VCProjectItems\newc++file.cpp` (administrator rights).

05 Assignment, compound assignment, ++ and --

▶ VIDEO 09:22

`=` stores the value on the right in the variable on the left. `c = b = a + 3;` works, but `b = a + 3; c = b;` is easier to read and debug.

CODE (N STARTS AT 10)	MEANS	N	M
`n += 5;`	`n = n + 5;` (also `-=` `*=` `/=` `%=`)	15	
`m = n++;`	use n, then add 1	11	**10**
`m = ++n;`	add 1, then use n	11	**11**
`m = n--;`	use n, then subtract 1	9	**10**
`m = --n;`	subtract 1, then use n	9	**9**

KEY IDEA

On its own, `n++;` and `++n;` do the same thing. The difference shows only when the value is used: in front, change first; behind, use first.

Unary `+` changes nothing (`+a` with `a = -5` is `-5`); unary `-` flips the sign (`-a` is `5`).

CODE 11 files on GitHub `github.com/abitclearer/c-programming-from-scratch/tree/main/15-arithmetic-operators`